

Reinforcement Learning

August · 09 Aug

chapter 1 - RL Problem

RL

- RL problems involve learning what to do, how to map situations to actions to maximize a numerical reward signal
- The learner is not told what actions to take but must figure out which actions yield the most results by testing them out.
- These actions may not affect the immediate reward but also the next situations and , through that, all subsequent results.
- The basic idea for a full specification of RL problems is simply to capture the most important aspects of the real problem facing a learning agent interacting with its environment to achieve a goal
- The agent must be able to 1) sense the environment to some extent 2) able to take some actions that affect the said environment. 3) have a goal or goals relating to the state of the environment.
- The formulation is intended to include these 3 aspects , sensation , action and goal, in their simplest possible form without trivializing any of them.
- RL is different from **supervised learning**, which is , learning from a training set of labeled examples provided by a knowledgeable external supervisor. Each example is a description of the situation together with a specific label, the label, of the correct action the system should take in that situation. The big is for system to extrapolate or generalize its responses so that it acts correctly in situations that are not present in the training set.

- It is often impractical to obtain examples of desired behaviour that are both correct and representative of all the situations in which the agent has to act.
- RL is also different from what is called **unsupervised learning**, which is typically about finding structure in collections of unlabeled data.
- Supervised learning and unsupervised learning appear to classify ML paradigms, but they do not. Although one may try to think of RL as a type of unsupervised learning as it also does not rely on examples of correct behaviour, but RL is trying to maximise the reward signal instead of trying to find a hidden structure.
- Uncovering the structure in an agent's experience can certainly be useful in RL but it does not address the RL agent's problem of maximising a reward signal.
- One of the challenges in RL is that of Exploration vs Exploitation. To obtain a lot of reward the agent must prefer actions it has taken in the past and found to have been effective in producing rewards. But to discover such actions, it has to take actions that it has not taken before.
- The agent has to **exploit** what it already knows in order to obtain rewards and it has to **explore** in order to make better actions in the future. Either exploration nor Exploitation can be taken exclusively without failing at the task. The agent must try a variety of actions and progressively favor that appear to be the best.
- Another key feature in RL is that it explicitly considers the WHOLE problem of a goal-directed agent interacting with an uncertain environment. This is in contrast with many approaches that consider subproblems without addressing how they fit into a larger picture.
- RL takes the opposite tack, starting with a complete, interactive goal-seeking agent. ALL RL agents have an explicit goal, sensation of the environment and action to influence the environment. It is usually assumed that the agent has to operate despite the significant uncertainty about the environment it faces. Explicit goal in a sense that the agent can judge progress towards its goal based on what it can sense directly.

- For planning, the agent has to address the interplay between the planning and the real-time action section, as well as question how the environment models are acquired and improved. When RL involves supervised learning it does so for a specific reasons that determines which capabilities are critical and which are not. Important subproblems should be isolated and studied but they should be subproblems that play a clear role in the complete, interactive, goal seeking agents even if all the details of the complete agent cannot yet be filled.

Elements of RL

Beyond the agent and the environment we can identify 4 main subelements of a RL system. a policy, a reward signal , a value function and optionally, a model of the environment.

Definition

A **policy** defines the learning agent's way of behaving at a given time. Roughly speaking, a **policy** is a mapping of perceived state of the environment to actions to be taken in those states.

In psychology this would be called a set of stimulus-response rule or associations. In some cases a policy may be a simple function or lookup table

The policy is the core of a reinforcement learning agent in the sense that it alone is sufficient to determine behavior. In general, policies may be stochastic.

Definition

A **reward signal** defines the goal in a RL problem. On each step , the environment sends to the RL agent a single number, a **reward**. The agent's sole objective is to maximize the total reward it receives in the long run. The reward signal thus defines what are the good and bad events for the agent.

The reward given to the agent at any given times depends on the agent's current action and the state of the agent's environment. The

agent cannot alter the process that does this. The only way that the agent can influence the reward signal is through its actions, which can have a direct impact on the reward or the an indirect effect through changing the environment's state.

The reward signal is the primary basis for altering the policy. If an action selected by the policy is followed by low reward, then the policy may be changed to select some other action in that situation in the future.

In general the reward signals may be stochastic functions of the state of the environment and the actions taken.

Definition

A **value function** specifies what is good in the long run. Roughly speaking, the **value** of a state is the total amount of rewards an agent can expect to accumulate over the future, starting from that state.

Whereas the rewards determine the immediate, intrinsic desirability of environment states, values indicate the long-term desirability of states after taking into account the the states that are likely to follow, and the rewards in that state. For example, a state might always yield a low immediate reward but still have a high value because it is regularly followed by other states that yield high rewards.

Rewards are in a sense primary, whereas values, as predictions of rewards, are secondary. Without rewards there could be no values, and the only purpose of estimating values is to achieve more reward.

It is values with which we are most concerned about when making and evaluating decisions. Action choices are made based on value judgements. We seeks actions that bring about highest values, not highest rewards because actions obtain the highest rewards for us over the long run.

In decision-making and planning, the derived quantity called value is the one with which we are most concerned. Unfortunately, it is much harder to determine values than it is to determine rewards.

Rewards are basically given directly by the environment, but values must be estimated and re-estimated from the sequences of observations an agent makes over its entire lifetime

Definition

A **model** of the environment is something that mimics the behavior of the environment, or more generally, that allow inferences to be made about how the environment will behave. For example, given a state and action, the model might predict the resultant next state and next reward.

Models are used to planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced.

RL methods that uses models and planning are called **model-based** methods.

Limitations and Scope

It is not strictly necessary to solve RL problems with methods centered around estimating the value functions.

For example, methods such as genetic algorithms, genetic programming, simulated annealing, and other optimization methods have been used to approach reinforcement learning problems without ever appealing to value functions. These methods evaluate the “life-time” behavior of many non-learning agents, each using a different policy for interacting with its environment, and select those that are able to obtain the most reward. We call these evolutionary methods because their operation is analogous to the way biological evolution produces organisms with skilled behavior even when they do not learn during their individual lifetimes. If the space of policies is sufficiently small, or can be structured so that good policies are common or easy to find, or if a lot of time is available for the search, then evolutionary methods can be effective. In addition, evolutionary methods have advantages on problems in which the learning agent cannot accurately sense the state of its environment.

Evolutionary methods ignore much of the useful structure of the reinforcement learning problem: they do not use the fact that the policy they are searching for is a function from states to actions; they do not notice which states an individual passes through during its lifetime, or which actions it selects

When we say that a reinforcement learning agent's goal is to maximize a numerical reward signal, we of course are not insisting that the agent has to actually achieve the goal of maximum reward. Trying to maximize a quantity does not mean that that quantity is ever maximized. The point is that a reinforcement learning agent is always trying to increase the amount of reward it receives. Many factors can prevent it from achieving the maximum, even if one exists. In other words, optimization is not the same as optimality.

2. Multi-arm Bandits

- The most important distinguishing feature of RL from other types of learning is that it uses information that **evaluates** the actions taken rather than **instruct** by giving correct decisions.
- purely evaluative feedback indicates how good the action taken is but not whether it is the best or worst action possible. Evaluative feedback is the basis of methods for function optimization, including evolutionary methods.
- Purely instructive feedback, on the other hand, indicates the correct action to take, independently of the action actually taken. This kind of feedback is the basis of supervised learning, which includes large parts of pattern classification, artificial neural networks, and system identification
- These two kinds of feedback seem to be quite distinct: evaluative feedback depends entirely on the action taken, whereas instructive feedback is independent of the action.
- A **nonassociative** setting is one in which prior work involving evaluative feedback has been done and it avoids much of the complexity of the full reinforcement learning problem.

2.1 AN n -armed Bandit problem

Consider the following learning problem. You are faced repeatedly with a choice among n different options, or actions. After each choice you receive a numerical reward chosen from a stationary probability distribution that depends on the action you selected. Your objective is to maximize the expected total reward over some time period, for example, over 1000 action selections, or time steps.

Each action has an expected or mean reward given that that action is selected; we call this the **value** of that action. If you knew the value of each action then it would be very trivial to solve the Problem: always select the action with the highest value. Thus, we assume you do not know the action values with certainty but you are free to make estimates.

If you maintain estimates of the action values, then at any time step there is at least one action whose estimated value is greatest. We call this a greedy action. If you select a greedy action, we say that you are exploiting your current knowledge of the values of the actions. If instead you select one of the nongreedy actions, then we say you are exploring, because this enables you to improve your estimate of the nongreedy action's value. Exploitation is the right thing to do to maximize the expected reward on the one step, but exploration may produce the greater total reward in the long run.

2.2 Action-value methods

We begin by looking more closely at some simple methods for estimating the values of actions and for using the estimates to make action selection decisions.

We denote the true (actual) value of action a as $q(a)$ and the estimated value of on the t -th time step as $Q_t(a)$

Recall that the true value of an action is the mean reward received when that action is selected.

One natural way of estimating this is by averaging the rewards when the action was selected. In other words, if by the t -th time step action

a has been chosen $N_t(a)$ times prior to t , yielding the rewards $R_1, R_2, \dots, R_{N_t(a)}$ then its value is estimated to be:

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)} \quad 0.1$$

if $N_t(a) = 0$ then we define $Q_t(a)$ instead as some default value, such as $Q_1(a) = 0$.

As $N_t(a) \rightarrow \infty$, $Q_t(a)$ converges to $q(a)$. We call this the **sample-average** method for estimating action-values.

Of course this is just one way to estimate the action value but not the best. The simplest action rule is to select the action with the highest estimated action value, that is, to select at time t one of the greedy actions, A_t^* for which $Q_t(A_t^*) = \max_a Q_t(A)$. This greedy action selection method can be written as:

$$A_t = \arg \max_a Q_t(a) \quad 0.2$$

where $\arg \max_a$ denotes the value of a at which the expression that follows is maximized (with ties broken arbitrarily). The greedy action selection method always exploits the current knowledge to maximise immediate reward; it spends no time at sampling apparently inferior actions to see if they might really be better. A simple alternative is to behave greedily most of the time but once in a while behave, say with probability of ϵ , instead to select randomly amongst all the actions with equal probability independent of the value estimates. We call methods using the near-greedy action selection rule as **ϵ -greedy** methods.

An advantage of these methods is that, in the limit as the number of plays increases, every action will be sampled an infinite number of times, guaranteeing that $N_t(a) \rightarrow \infty$ for all a and thus ensuring that all $Q_t(a)$ converge to $q(a)$. This implies that the probability of selecting the optimal action converges to greater than $1 - \epsilon$.

Another thing to note is what to do when the multiple actions have the same maximal action values. Such ties can be broken randomly. An alternative that has the same effect is to add a very small amount of

randomness to each of the initial actions values , so that ties effectively never happen.

Finally, The advantage of ϵ -greedy over greedy methods depends on the task. For example, suppose the reward variance had been larger, say 10 instead of 1. With noisier rewards it takes more exploration to find the optimal action, and ϵ -greedy methods should fare even better relative to the greedy method. On the other hand, if the reward variances were zero, then the greedy method would know the true value of each action after trying it once. In this case the greedy method might actually perform best because it would soon find the optimal action and then never explore. But even in the deterministic case, there is a large advantage to exploring if we weaken some of the other assumptions. For example, suppose the bandit task were nonstationary, that is, that the true values of the actions changed over time. In this case exploration is needed even in the deterministic case to make sure one of the nongreedy actions has not changed to become better than the greedy one.

2.3 Incremental Implementation

THE Action-value method that was discussed above, estimate actions value by averaging the observed rewards. Hence, the obvious Implementation is to maintain, for each action a , a record of all the rewards that have followed the selection of that action. Then the estimated of the value of action a at time t can be computed as:

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)} \quad 0.3$$

A problem with this strategy is that its memory and computational requirements grow over time without bound.

It is easy to devise incremental update formulas for computing averages with small, constant computation required to process each new reward.

For some action a , let Q_k denote the estimate for its k th reward, that is the average of the first $k - 1$ rewards. Given this average and the k th

reward for the action, the average of the all the k -rewards can be computed by

$$\begin{aligned}
 Q_{k+1} &= \frac{1}{k} \sum_{i=1}^k R_i \\
 &= \frac{1}{k} \left(R_k + \sum_{i=1}^{k-1} R_i \right) \\
 &= \frac{1}{k} (R_k + (k-1)Q_k + Q_k - Q_k) \\
 &= \frac{1}{k} (R_k + kQ_k - Q_k) \\
 &= Q_k + \frac{1}{k} (R_k - Q_k)
 \end{aligned}
 \tag{0.4}$$

This implementation requires memory for Q_k and k .

This is a update rule and has the form:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{stepSize} [\text{Target} - \text{OldEstimate}] \tag{0.5}$$

The expression $[\text{Target} - \text{OldEstimate}]$ is the error in the estimate. It is reduced by taking a step toward the "Target." The target is presumed to indicate a desirable direction in which to move, though it may be noisy. In the case above, for example, the target is the k th reward.

The step size parameter used in the incremental method described above changes from time step to time step. We denote the step-size parameter as α or more generally as $\alpha_{t(a)}$, in this case, $\alpha = \frac{1}{k}$

2.4 Tracking a nonstationary Problem.

The averaging method discussed so far are appropriate in a stationary environment, but not if the bandit's actual value is changing over time. In such cases, it makes sense to weight in recent rewards more heavily than long past ones. One of the most popular ways of doing this is to use a constant step-size parameter. For example,

$$Q_{k+1} = Q_k + \alpha [R_k - Q_k] \tag{0.6}$$

where the step size parameter $\alpha \in (0,1]$ is constant. This results in Q_{k+1} being the weighted average of the past rewards and the initial estimate.

$$\begin{aligned}
 Q_k &= Q_k + \alpha[R_k - Q_k] \\
 &= \alpha R_k + (1 - \alpha)Q_k \\
 &= \alpha R_k + (1 - \alpha)[\alpha R_{k-1} + (1 - \alpha)Q_{k-1}] \\
 &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 Q_{k-1} & 0.7 \\
 &= \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 \alpha R_{k-2} + \dots + (1 - \alpha)^{k-1} \alpha R_1 + (1 - \alpha)^k Q_1 \\
 &= (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i
 \end{aligned}$$

This called the weighted average because the sum of the weights $(1 - \alpha)^k + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} = 1$. The weight given to R_i decreases as the number of intervening rewards increases. In fact, the weight decreases exponential as the according to the exponent on $1 - \alpha$, (if $1 - \alpha = 0$ then all the weight goes on the very last reward, R_k because of the convention that $0^0 = 1$). This assumption is called **exponential, recency-weighted average**

Sometimes it is convenient to vary the step size parameter from step to step. Let $\alpha_k(t)$ denote the step-size parameter used to process the reward received after the k th reward selection of $\mathbf{a}$. As notes, $\alpha_k(t) = \frac{1}{k}$ results in the sample-average method, which is guaranteed to converge to the true action values by the law of large numbers.

But of course convergence is not guaranteed for all choices of the sequence $\{\alpha_k(\mathbf{a})\}$. A well-known result in stochastic approximation theory gives us the conditions required to assure convergence with probability 1:

$$\sum_{k=1}^{\infty} \alpha_{k(\mathbf{a})} = \infty \text{ and } \sum_{k=1}^{\infty} \alpha_k^2(\mathbf{a}) < \infty \quad 0.8$$

The first condition is required to guarantee that the steps are large enough to eventually overcome any initial conditions or random

fluctuations. The second condition guarantees that eventually the steps become small enough to assure convergence.

Note that both conditions are met when $\alpha_{k(a)} = \frac{1}{k}$ but not for the constant parameter $\alpha_{k(a)} = \alpha$. In the latter case, the second condition is not met indicating that the estimates never completely converge but continue to vary in response to the most recently received reward.

As we mentioned above, this is actually desirable in a nonstationary environment, and problems that are effectively nonstationary are the norm in reinforcement learning.

Sequences that meet the two conditions often converge very slowly or need tuning in order to obtain a satisfactory convergence rate.

2.5 Optimistic Initial values

All the methods we have discussed so far are dependent to some extent on the initial action-value estimates, $Q_1(a)$. In the language of statistics, these methods are biased by their initial estimates. For the sample-average methods, the bias disappears once all actions have been selected at least once, but for methods with constant α , the bias is permanent, though decreasing over time. The bias are usually not a problem and can be very useful but the downside is that the initial estimates, becomes, in effect a set of parameters that must be picked by the user.

Initial action values can also be used as a simple way of encouraging exploration. We call this technique, **Optimistic Initial Values**. We regard it as a simple trick that can be quite effective on stationary problems, but it is far from being a generally useful approach to encouraging exploration. For example, it is not well suited to nonstationary problems because its drive for exploration is inherently temporary. If the task changes, creating a renewed need for exploration, this method cannot help. Indeed, any method that focuses on the initial state in any special way is unlikely to help with the general nonstationary case. The beginning of time occurs only once, and thus we should not focus on it too much. This criticism applies as well to the sample-average methods,

which also treat the beginning of time as a special event, averaging all subsequent rewards with equal weight.

2.6 Upper-Confidence-Bound Action selection

Exploration is needed because the estimates of the action values are uncertain. The greedy actions are those that look best at present, but some of the other actions may actually be better. ϵ -greedy action selection forces the non-greedy actions to be tried, but indiscriminately, with no preference for those that are nearly greedy or particularly uncertain. It would be better to select among the non-greedy actions according to their potential for actually being optimal, taking into account both how close their estimates are to being maximal and the uncertainties in those estimates. One effective way of doing this is to select actions as

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right] \quad 0.9$$

if $N_t(a) = 0$ then a is considered to be a maximizing action.

The idea of this **upper confidence bound** (UCB) action selection is that the square-root term is a measure of the uncertainty or variance in the estimate of a 's value. The quantity being max'ed over is thus a sort of upper bound on the possible true value of action a , with the c parameter determining the confidence level. Each time a is selected the uncertainty is presumably reduced; $N_t(a)$ increases and the uncertainty measure term is decreased. On the other hand, each time that action other a is selected, t is increased and the uncertainty estimate increases too. The natural log makes the increase a bit smaller over time but is unbounded; all actions will eventually be selected, but as time goes by it will be a longer wait and thus a lower selection frequency, for actions with a lower value-estimate or that have already been selected more times.

UCB will often perform well, as shown here, but is more difficult than ϵ -greedy to extend beyond bandits to the more general reinforcement

learning settings general reinforcement learning settings considered in the rest of this book. One difficulty is in dealing with nonstationary problems; something more complex than the methods presented in Section 2.4 would be needed. Another difficulty is dealing with large state spaces, particularly function approximation.

2.7 Gradient Bandits

Instead of considered methods that estimate action values and use those estimates to select actions; in this section, we consider learning **numerical preference $H_t(a)$** for each action a . The larger the preference, the more often the action is taken but the preference has no interpretation in terms of the rewards. Only the relative preference of one action over another is important; if we add 1000 to all the preferences there is no affect on the action probabilities, which are determined according to a soft-max distribution (i.e., Gibbs or Boltzmann distribution) as follows:

$$\Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^n e^{H_t(b)}} = \pi_t(a) \quad 0.10$$

$\pi_t(a)$ is a new notation for the probability of taking action a at time t . Initially are preferences are the same, $H_1(a) = 1 \quad \forall a$ so that all actions have equal probability of being selected.

A natural learning algorithm for this setting based on the idea of stochastic gradient ascent; on each step, after selecting the action A_t and receiving the reward R_t , the preferences are updated by:

$$\begin{aligned} H_{t+1} &= H_{t(A_t)} + \alpha(R_t - \bar{R}_t(1 - \pi_t(A_t))). \quad \text{and} \\ H_{t+1} &= H_{t(a)} - \alpha(R_t - \bar{R}_t)\pi_t(a) \end{aligned} \quad 0.11$$

where $\alpha > 0$ is a step size param, and $\bar{R}_t \in \mathbb{R}$ is the average of all the rewards up through and including t , which can be computed incrementally using the methods in section 2.3 or 2.4 if the problem is nonstationary. The $\bar{R}_t$ term serves as a baseline with which the reward is compared. If the reward is higher than the baseline, then the probability of taking A_t At in the future is increased, and if the reward

is below baseline, then probability is decreased. The non-selected actions move in the opposite direction.

One can gain a deeper insight into this algorithm by understanding it as a stochastic approximation to gradient ascent. In exact **gradient ascent**, each preference $H_t(a)$ would be incrementing proportional to the increment's effect on performance:

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)}. \quad 0.12$$

where the measure of the increment here is the expected reward.

$$\mathbb{E}[R_t] = \sum_b \pi_t(b) q(b) \quad 0.13$$

It is not possible to implement gradient ascent exactly in our case we do not know the $q(b)$ but in fact the updates of our algorithm 0.11 are equal to 0.12 in expected value, making the algorithm an instance of **stochastic gradient ascent**:

Proof.

$$\begin{aligned} \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} &= \frac{\partial}{\partial H_t(a)} \left[\sum_b \pi_t(b) q(b) \right] \\ &= \sum_b q(b) \frac{\partial \pi_t(b)}{\partial H_t(a)} \\ &= \sum_b (q(b) - x_t) \frac{\partial \pi_t(b)}{\partial H_t(a)} \end{aligned}$$

where x_t can be any scalar that does not depend on b , we can include it here because the gradient sum to zero over all the actions, $\sum_b \frac{\partial \pi_t(b)}{\partial H_t(a)} = 0$. As H_t is changed, some actions' probabilities go up and some go down but the sum of changes must be zero because the sum of the probabilities remain 1.

$$= \sum_b \pi_t(b) (q(b) - x_t) \frac{\partial \pi_t(b)}{\partial H_t(a)} / \pi_t(b)$$

The equation is now in the form of an expectation, summing over all possible values b of the random variable A_t , then multiplying by the probability of taking those values. Thus:

$$\begin{aligned} &= \mathbb{E}[(q(A_t) - x_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t)] \\ &= \mathbb{E}[(R_t - \bar{R}_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t)] \end{aligned}$$

we chose $x_t = R_t$ and substitute R_t for $q(A_t)$, which is permitted because $\mathbb{E}(R_t) = q(A_t)$ because all the other factors are non random.

We later show that

$$\frac{\partial \pi_t(b)}{\partial H_t(a)} = \pi_t(b)(\mathbb{I}_{a=b} - \pi_t(a)) \quad 0.14$$

where $\mathbb{I}_{a=b} = 1$ if $a = b$, else 0

Assuming that for now we have:

$$\begin{aligned} &= \mathbb{E}[(R_t - \bar{R}_t) \pi_t(A_t)(\mathbb{I}_{a=A_t} - \pi_t(a)) / \pi_t(A_t)] \\ &= \mathbb{E}[(R_t - \bar{R}_t)(\mathbb{I}_{a=A_t} - \pi_t(a))] \end{aligned} \quad 0.15$$

Our plan has been to write the performance gradient as an expectation of something that we can sample on each step, as we have just done, and then update on each step proportional to the sample.

Substituting a sample of the expectation above for the performance gradient, yields :

$$H_{t+1} = H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{I}_{a=A_t} - \pi_t(a)), \quad \forall a \quad 0.16$$

Which is equivalent to the original algorithm.

We now show that:

$$\frac{\partial \pi_t(b)}{\partial H_t(a)} = \pi_t(b)(\mathbb{I}_{a=b} - \pi_t(a)) \quad 0.17$$



Associative Search

So far in this chapter we have considered only nonassociative tasks, in which there is no need to associate different actions with different situations. In these tasks the learner either tries to find a single best action when the task is stationary, or tries to track the best action as it changes over time when the task is nonstationary. However in general RL task, there is more than one situation and the goal is to learn a **policy**: a mapping from situations to the actions that are best in those situations.

Here is a simple way in which nonassociative tasks extend to associative setting. Suppose there are several different n -armed bandit tasks, and that on each play you confront one of these chosen at random. Thus, the bandit task changes randomly from play to play. This would appear to you as a single, nonstationary n -armed bandit task whose true action values change randomly from play to play. You could try using one of the methods described in this chapter that can handle nonstationarity, but unless the true action values change slowly, these methods will not work very well. Now suppose, however, that when a bandit task is selected for you, you are given some distinctive clue about its identity (but not its action values). Maybe you are facing an actual slot machine that changes the color of its display as it changes its action values. Now you can learn a policy associating each task, signaled by the color you see, with the best action to take when facing that task—for instance, if red, play arm 1; if green, play arm 2. With the right policy you can usually do much better than you could in the absence of any information distinguishing one bandit task from another.

3. Finite Markov Decision

3.1 The agent's environment Interface .

The reinforcement learning problem is meant to be a straightforward framing of the problem of learning from interaction to achieve a goal. The learner and decision-maker is called the agent. The thing it interacts with, comprising everything outside the agent, is called the

environment. These interact continually, the agent selecting actions and the environment responding to those actions and presenting new situations to the agent. The environment gives rises rewards, that the agent tries to maximize.

The agent and the environment interact at each of a sequence of discrete time steps, $t = 1, 2, \dots$. At each time step, the agent receives some representation of the environment's state $S_t \in \mathcal{S}$, where $\mathcal{S}$ is a set of possible states, and on that Basis selects an action, $A_t \in \mathcal{A}(S_t)$ where $\mathcal{A}(S_t)$ is a set of actions available in set state S_t . One time step, in part as a consequence of its action, the agent receives a numerical **reward**, $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ and finds itself in a new different state.

At each time step, the agent implements a mapping from states to probabilities of selecting each possible action. This mapping is called the agent's policy and is denoted as π_t , where $\pi_t(a | s)$ is the probability that $A_t = a$ if $S_t = s$. RL methods specifies how the agent changes its policy as a result of its experience. The agent's goal, roughly speaking, is to maximize the total amount of reward it receives over the long run.

What we chose as part of the environment is particularly up to us, but The general rule we follow is that anything that cannot be changed arbitrarily by the agent is considered to be outside of it and thus part of its environment. We do not assume that everything in the environment is unknown to the agent. For example, the agent often knows quite a bit about how its rewards are computed as a function of its actions and the states in which they are taken. But we always consider the reward computation to be external to the agent because it defines the task facing the agent and thus must be beyond its ability to change arbitrarily. In fact, in some cases the agent may know everything about how its environment works and still face a difficult reinforcement learning task, just as we may know exactly how a puzzle like Rubik's cube works, but still be unable to solve it. The agent-environment boundary represents the limit of the agent's absolute control, not of its knowledge.

3.2 Goals and Rewards

In RL, the goal or the purpose of the agent is formalized in terms of reward signals, the agent's goal is to maximize the total amount of reward not the immediate reward. We can clearly state this informal idea as the **reward hypothesis**:

That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).

If we want the agent to do something for us, we must reward the points in such a way that maximizing the rewards, the agent will achieve our goal. It is thus critical that the rewards we set up truly indicate what we want accomplished. In particular, the reward signal is not the place to impart to the agent prior knowledge about how to achieve what we want it to do.⁴ For example, a chess-playing agent should be rewarded only for actually winning, not for achieving subgoals such as taking its opponent's pieces or gaining control of the center of the board. If achieving these sorts of subgoals were rewarded, then the agent might find a way to achieve them without achieving the real goal. For example, it might find a way to take the opponent's pieces even at the cost of losing the game. The reward signal is your way of communicating to the robot what you want it to achieve, not how you want it achieved.

3.3 Returns

How do we define the concept of maximizing the rewards formally? If the sequence of rewards received after time step t is denoted $R_{t+1}, R_{t+2}, R_{t+3}, \dots$ then what precise aspect of this sequence do we wish to maximize? In general, we seek to maximize the expected return, where the return G_t is specified as some specific function of the reward sequence. In the simplest case, it is the sum of the rewards,

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T \quad 0.1$$

where T is the final time step. This approach makes sense in applications in which there is a natural notion of final time step, that

is, when the agent environment interaction breaks naturally into subsequences, which we call episodes, such as plays of a game, trips through a maze, or any sort of repeated interactions. Each episode ends in a special state called the terminal state, followed by a reset to a standard starting state or to a sample from a standard distribution of starting states. Tasks with episodes of this kind are called episodic tasks. In episodic tasks we sometimes need to distinguish the set of all nonterminal states, denoted $\mathcal{S}$, from the set of all states plus the terminal state, denoted $\mathcal{S}^+$

On the other hand, in many cases the agent–environment interaction does not break naturally into identifiable episodes, but goes on continually without limit. For example, this would be the natural way to formulate a continual process-control task, or an application to a robot with a long life span. We call these continuing tasks. The sum return formulation is a bit problematic in this case as the final time step, $T = \infty$, and the return which we are in turn trying to maximize could be infinite

The additional concept that we need is that of discounting. According to this approach, the agent tries to select actions so that the sum of the discounted rewards it receives over the future is maximized. In particular it chooses A_t to maximize the expected **discounted return**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad 0.2$$

where γ is a parameter $0 \leq \gamma \leq 1$, called the discount rate.

The discount rate determines the present value of future rewards: a reward received k time steps in the future is worth only γ^{k-1} times it would be worth if it were received immediately. If $\gamma < 1$, the infinite sum has a value as long as the sequence $\{R_k\}$ is bounded. If $\gamma = 0$ the agent is “myopic” in being concerned only with maximizing immediate rewards: its objective in this case is to learn how to choose A_t so as to maximize only R_{t+1}

If each of the agent’s actions happened to influence only the immediate reward, not future rewards as well, then a myopic agent could maximize

(3.2) by separately maximizing each immediate reward. But in general, acting to maximize immediate reward can reduce access to future rewards so that the return may actually be reduced. As γ approaches 1, the objective takes future rewards into account more strongly; the agent becomes more farsighted.

3.4 Unified Notation for Episodic and continuing Tasks

We establish a notation to speak of the agent-environment that breaks down into sequence of separate episodes and one in which it does not. To be precise about episodic tasks requires some additional notation.

Let $S_{t,i}$ the state representation of at time t in episode i , and similarly for $A_{t,i}$, $R_{t,i}$, $\pi_{t,i}$, T , etc. However, it turns out that, when we discuss episodic tasks we will almost never have to distinguish between different episodes. We will almost always be considering a particular single episode, or stating something that is true for all episodes. So, in practice we can abuse the notation and drop the subscript i .

We need one other convention to obtain a single notation that covers both episodic and continuing tasks. We have defined the return as a sum over a finite number of terms in one case (3.1) and as a sum over an infinite number of terms in the other (3.2). These can be unified by considering episode termination to be the entering of a special absorbing state that transitions only to itself and that generates only rewards of zero.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \quad 0.3$$

We use these conventions throughout the rest of the book to simplify notation and to express the close parallels between episodic and continuing tasks.

3.5 Markov Property

In the reinforcement learning framework, the agent makes its decisions as a function of a signal from the environment called the environment's

state. In this section we discuss what is required of the state signal, and what kind of information we should and should not expect it to provide. In particular, we formally define a property of environments and their state signals that is of particular interest, called the Markov property.

The state is simply whatever information the agent has to act on, produced by some preprocessing step the book treats as part of the environment rather than something to be designed or learned, the focus is on choosing actions given the signal, not on building it. That signal can go well beyond raw sensation: it may be a processed form of current input, or something accumulated over time (a scene assembled across eye movements, an object remembered after looking away, the meaning of “yes” depending on the question that preceded it, velocity derived from two position readings). But it also shouldn’t be expected to carry everything relevant, the next card in the deck, the caller’s identity, an unconscious patient’s internal injuries are all hidden from the agent because no sensation ever revealed them. The standard is memory, not omniscience: the agent isn’t blamed for never having learned something, only for having received the information and then lost it.

What we would like, ideally, is a state signal that summarizes past sensations compactly, yet in such a way that all relevant information is retained. This normally requires more than the immediate sensations, but never more than the complete history of all past sensations. A state signal that succeeds in retaining all relevant information is said to be Markov, or to have the Markov property (we define this formally below). e.g. checkers position, the current config of the board is a Markov property, and for a cannonball, the current position and velocity is all that matter for its future flights, it does not matter how that current position or velocity came to be. Sometimes called independence of paths.

We now formally define the Markov property for the reinforcement learning problem. To keep the mathematics simple, we assume here that there are a finite number of states and reward values. This enables us to work in terms of sums and probabilities rather than integrals and

probability densities, but the argument can easily be extended to include continuous states and rewards. Consider how a general environment might respond at time $t + 1$ to the action taken at time t . In the most general, causal case this response may depend on everything that has happened earlier. In this case the dynamics can be defined only by specifying the complete probability distribution:

$$\Pr\{R_{t+1} = r, S_{t+1} = s' \mid S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A - t\} \quad 0.4$$

for all r, s' and possible values of the past events,

$$S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A - t$$

If the state signal has the Markov property, on the other hand, then the environment's response at $t + 1$ depends only on the state and action representations at t , in which case the environment's dynamics can be defined by specifying only:

$$\Pr(s', r \mid s, a) = \Pr\{R_{t+1} = r, S_{t+1} = s' \mid S_t, A_t\} \quad 0.5$$

In other words, a state signal has the Markov property, and is a Markov state, if and only if (3.5) is equal to (3.4) for all s', r , and histories $S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A - t$

In this case, the environment and task as a whole are also said to have the Markov property. We always want the state to be a good basis for predicting future rewards and for selecting actions. In cases in which a model of the environment is learned (see Chapter 8), we also want the state to be a good basis for predicting subsequent states. Markov states provide an unsurpassed basis for doing all of these things.

To the extent that the state approaches the ability of Markov states in these ways, one will obtain better performance from reinforcement learning systems. For all of these reasons, it is useful to think of the state at each time step as an approximation to a Markov state, although one should remember that it may not fully satisfy the Markov property.

The Markov property is important in reinforcement learning because decisions and values are assumed to be a function only of the current state.